

```
points."
estimated_request_tokens = count_tokens(prompt)

response = client.responses.create(
    model="gpt-4o-mini",
    input=prompt
)

actual_usage = response.usage
print("Estimated request tokens:", estimated_request_tokens)
print("Actual input tokens:", actual_usage.input_tokens)
print("Actual output tokens:", actual_usage.output_tokens)
print("Total tokens:", actual_usage.total_tokens)
```

Local tokenizers such as tiktoken are useful for fast estimation, especially for plain text. API-provided usage values are more authoritative because they include model-specific formatting and execution details. A robust framework should store both values: the estimate for preflight checks and the actual usage for billing, reporting, and continuous improvement.

Framework packaging and open source readiness

To make the framework useful to the community, the project should be packaged cleanly and documented well. A recommended structure is:

```
promptopt/
  optimizer.py
  token_counter.py
  semantic.py
  validators.py
  cli.py
tests/
  test_optimizer.py
examples/
  basic_usage.py
pyproject.toml
README.md
```

The command-line interface can make the tool easy to use:

```
promptopt optimize --input prompt.txt --model gpt-4o-mini
--mode safe

The Python API can be equally simple:
from promptopt import PromptOptimizer

optimizer = PromptOptimizer(model="gpt-4o-mini", mode="safe")
result = optimizer.optimize(prompt)
```

```
print(result["optimized_prompt"])
```

Validation: Ensuring the prompt still works

Token reduction is useful only if the optimized prompt preserves functionality. The framework should therefore validate each optimized prompt using three checks. First, it should confirm that mandatory intent phrases are still present. Second, it should compare the semantic similarity of the original and optimized prompt. Third, it should optionally run both prompts against a test set and compare outputs using quality metrics or human review. This creates an engineering discipline around prompt compression rather than treating it as a blind text-shortening exercise.

Here are a few best practices:

- Start with safe optimization rules before enabling aggressive semantic compression.
- Allow users to mark protected text that must never be changed.
- Measure token count before and after optimization for every prompt.
- Track actual API usage after execution, not just estimated tokens.
- Maintain test prompts for each application so that optimization does not silently degrade output quality.
- Expose configuration options such as compression level, similarity threshold, and model name.
- Keep logs of original prompt, optimized prompt, token savings, and validation status for auditability.

An open source token optimization framework for prompt development can bring discipline, repeatability, and cost awareness to LLM application engineering. By combining rule-based cleanup, duplicate intent removal, semantic deduplication, protected sections, token counting, and post-execution usage monitoring, developers can reduce prompt size without losing meaning. The most important principle is balance. A good optimizer should not simply make text shorter; it should make prompts cleaner, more economical, and easier for both humans and models to understand. **END** 

 By: Dr Magesh Kasthuri and Dr Anand Nayyar

Dr Magesh Kasthuri is a PhD in artificial intelligence and the genetic algorithm. He currently works as a distinguished member of the technical staff (master) and chief architect at Wipro Ltd. This article expresses his views and not of the organisation he works in.

Dr Anand Nayyar is a PhD in wireless sensor networks and swarm intelligence. He works at Duy Tan University, Vietnam, and loves to explore open source technologies, IoT, cloud computing, deep learning, and cyber security.